PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
int data;
struct Node* next;
};
struct Node* head = NULL;
// Function to create a new node
struct Node* createNode(int data) {
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
newNode->data = data;
newNode->next = NULL;
return newNode;
}
// Function to insert a node at the beginning
void insertAtBeginning(int data) {
struct Node* newNode = createNode(data);
newNode->next = head;
head = newNode;
}
// Function to insert a node at the end
void insertAtEnd(int data) {
struct Node* newNode = createNode(data);
if (head == NULL) {
head = newNode;
return;
}
struct Node* temp = head;
while (temp->next != NULL) {
temp = temp->next;
}
temp->next = newNode;
}
// Function to insert a node at a specified position
void insertAtPosition(int data, int position) {
if (position < 1) {
printf("Invalid position\n");
return;
}
if (position == 1) {
insertAtBeginning(data);
return;
```

```
}
struct Node* newNode = createNode(data);
struct Node* temp = head;
int count = 1;
while (count < position - 1 && temp != NULL) {
temp = temp->next;
count++;
}
if (temp == NULL) {
printf("Invalid position\n");
return;
}
newNode->next = temp->next;
temp->next = newNode;
}
// Function to delete a node from the beginning
void deleteAtBeginning() {
if (head == NULL) {
printf("List is empty\n");
return;
}
struct Node* temp = head;
head = head->next;
free(temp);
}
// Function to delete a node from the end
void deleteAtEnd() {
if (head == NULL) {
printf("List is empty\n");
return;
}
if (head->next == NULL) {
free(head);
head = NULL;
return;
}
struct Node* temp = head;
while (temp->next->next != NULL) {
temp = temp->next;
}
free(temp->next);
temp->next = NULL;
}
// Function to delete a node at a specified position
```

```
void deleteAtPosition(int position) {
if (position < 1) {
printf("Invalid position\n");
return;
}
if (position == 1) {
deleteAtBeginning();
return;
}
struct Node* temp = head;
int count = 1;
while (count < position - 1 && temp != NULL && temp->next != NULL) {
temp = temp->next;
count++;
}
if (temp == NULL || temp->next == NULL) {
printf("Invalid position\n");
return;
}
struct Node* toDelete = temp->next;
temp->next = toDelete->next;
free(toDelete);
}
// Function to display the linked list
void display() {
if (head == NULL) {
printf("List is empty\n");
return;
}
struct Node* temp = head;
while (temp != NULL) {
printf("%d -> ", temp->data);
temp = temp->next;
}
printf("NULL\n");
}
int main() {
int choice, data, position;
while (1) {
printf("\nSingly Linked List Operations:\n");
printf("1. Insert at Beginning\n");
printf("2. Insert at End\n");
printf("3. Insert at Position\n");
printf("4. Delete at Beginning\n");
```

```
printf("5. Delete at End\n");
printf("6. Delete at Position\n");
printf("7. Display\n");
printf("8. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice) {
case 1:
printf("Enter data to insert at beginning: ");
scanf("%d", &data);
insertAtBeginning(data);
break;
case 2:
printf("Enter data to insert at end: ");
scanf("%d", &data);
insertAtEnd(data);
break;
case 3:
printf("Enter data to insert: ");
scanf("%d", &data);
printf("Enter position: ");
scanf("%d", &position);
insertAtPosition(data, position);
break;
case 4:
deleteAtBeginning();
break;
case 5:
deleteAtEnd();
break;
case 6:
printf("Enter position to delete: ");
scanf("%d", &position);
deleteAtPosition(position);
break;
case 7:
display();
break;
case 8:
exit(0);
default:
printf("Invalid choice\n");
}
}
```

```
return 0;
}
```